

Ollama CLI Commands

Ollama is a platform that enables users to run Large Language Models (LLMs) directly on their local machine. It provides simple commands to download, manage, and interact with AI models without requiring cloud services.

Model Download and Usage Commands

1. **ollama pull llama3**

Downloads the Llama 3 model from the Ollama library and saves it locally so it can be used without an internet connection.

2. **ollama list**

Displays all AI models currently installed on the system.

3. **ollama show llama3**

Provides detailed information about the selected model, including size, architecture, parameters, and other technical details.

4. **ollama run llama3**

Starts the Llama 3 model and opens an interactive environment where users can enter prompts and receive AI-generated responses.

5. **ollama run llama3 --verbose**

Runs the model in verbose mode, showing additional information such as token generation rate, processing time, and performance metrics.

Interactive Session Commands

/help

Displays a list of available commands that can be used during an active chat session.

/bye

Ends the current conversation and exits the chat interface.

/show

Shows the current model configuration, settings, and active parameters.

/set

Lists all adjustable parameters that can be customized for the running model.

/set parameter temperature 0.8

Changes the temperature value, affecting the randomness and creativity of generated responses.

/set system "You are an AI tutor"

Defines a system instruction that influences the model's behavior throughout the conversation.

Model Monitoring and Management

->ollama ps

Displays all models currently loaded in memory along with their running status.

->ollama stop llama3

Stops the specified model and frees the memory and processing resources it was using.

->ollama rm llama3

Removes the downloaded model from local storage to free disk space.

```
(c) Microsoft Corporation. All rights reserved.

C:\Users\DORERAJ>ollama pull llama3
pulling manifest
pulling 6a0746alec1a: 100% 4.7 GB
pulling 4fa551d4f938: 100% 12 KB
pulling 8ab4849b038c: 100% 254 B
pulling 577073ffcc6c: 100% 110 B
pulling 3f8eb4da87fa: 100% 485 B
verifying sha256 digest
writing manifest
success

C:\Users\DORERAJ>ollama ls
NAME          ID          SIZE      MODIFIED
llama3:latest  365c0bd3c000 4.7 GB    5 seconds ago
llama3.2:latest a80c4f17acd5 2.0 GB    13 hours ago
mistral:latest 6577803aa9a0 4.4 GB    23 hours ago

C:\Users\DORERAJ>ollama show mistral
Model
  architecture llama
  parameters    7.2B
  context length 32768
  embedding length 4096
  quantization  Q4_K_M

Capabilities
  completion
  tools

Parameters
  stop "[INST]"
  stop "[/INST]"

License
  Apache License
  Version 2.0, January 2004
  ...
```

```
C:\Users\DORERAJ>ollama run mistral
>>> hlo
Hello! How can I help you today? If you have any questions
or need assistance, feel free to ask.

Here are some ways I can assist you:

1. Answering questions about a wide range of topics (such as
technology, science, history, culture, and more)
2. Providing explanations for complex concepts
3. Helping with homework or study materials
4. Generating ideas or brainstorming solutions to problems
5. Offering recommendations for books, movies, TV shows,
music, and other forms of entertainment
6. Assisting with writing and grammar questions
7. Providing fun facts, trivia, and interesting information on various topics
8. Offering friendly conversation and being a companion when you need someone to talk to.

Please let me know if there's anything specific you'd like help with, or if you have any other requests! I'm here to make your experience as enjoyable
and helpful as possible.

>>> /bye

C:\Users\DORERAJ>ollama serve
Error: listen tcp 127.0.0.1:11434: bind: Only one usage of each socket address (protocol/network address/port) is normally permitted.

C:\Users\DORERAJ>ollama stop mistral

C:\Users\DORERAJ>ollama ps
NAME    ID    SIZE    PROCESSOR    CONTEXT    UNTIL
C:\Users\DORERAJ>
```

Testing Ollama API Using Postman

Postman is an API testing tool used to send requests to the Ollama server and view the responses generated by the AI model.

To test the Generate API, send a POST request to:

`http://localhost:11434/api/generate`

Use the following JSON body:

```
{  
  "model": "llama3",  
  "prompt": "Why is the sky blue?"  
}
```

When the request is sent, Ollama processes the prompt using the specified model and returns an AI-generated response in JSON format.

Purpose:

- Test whether the Ollama server is running correctly.
- Verify that the model is generating responses.
- Check API integration before using it in applications.

Output:

The API returns the generated text response along with additional information such as model name and generation details.

My Collection > Test API

POST http://localhost:11434/api/generate Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "model": "mistral",
3   "prompt": "What is ai"
4 }
```

Body Cookies Headers 3 Test Results 200 OK • 20.68 s • 12.55 KB Save Response

{ JSON Preview Visualize

```
1 {
2   "model": "mistral",
3   "created_at": "2026-06-12T05:32:58.8465398Z",
4   "response": " AI",
5   "done": false
6 }
7 {
8   "model": "mistral",
9   "created_at": "2026-06-12T05:32:58.974581Z",
10  "response": ", ",
11  "done": false
12 }
13 {
14  "model": "mistral",
```

My Collection > Test API

POST http://localhost:11434/api/generate Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "model": "llama3",
3   "prompt": "What is ai"
4 }
```

Body Cookies Headers 3 Test Results 200 OK • 1 m 33.36 s • 46.37 KB Save Response

{ JSON Preview Visualize

```
1 {
2   "model": "llama3",
3   "created_at": "2026-06-12T05:36:24.1332761Z",
4   "response": "AI",
5   "done": false
6 }
7 {
8   "model": "llama3",
9   "created_at": "2026-06-12T05:36:24.2623431Z",
10  "response": " (",
11  "done": false
12 }
13 {
14  "model": "llama3",
```

Testing Ollama API Using Python Requests Library in IDLE

Definition:

Testing the Ollama API using Python's Requests library in IDLE enables communication with locally installed AI models through HTTP requests.

Purpose

The purpose of testing the Ollama API is to:

- Verify that Ollama is installed and running correctly.
- Ensure the AI model can receive and process prompts.
- Learn how Python applications interact with APIs.
- Understand the basics of AI model integration.
- Build applications such as chatbots, virtual assistants, and automation tools.

Working Process

1. The Python program imports the Requests library.
2. The API endpoint (`http://localhost:11434/api/generate`) is specified.
3. A JSON payload containing the model name and prompt is created.
4. Python sends a POST request to the Ollama API.
5. Ollama receives the request and forwards the prompt to the selected AI model.
6. The AI model processes the prompt and generates a response.
7. Ollama returns the response in JSON format.
8. Python extracts the generated text and displays it in IDLE.

Benefits

- Simple and easy-to-use API integration.
- Supports locally running AI models.
- No internet connection required after model download.
- Faster response times compared to cloud-based services.

```
python test86.py - C:/Users/DORERAJ/AppData/Local/Programs/Python/Python313/python...
File Edit Format Run Options Window Help
import requests

url = "http://localhost:11434/api/generate"

data = {
    "model": "llama3.2",
    "prompt": "Hello",
    "stream": False
}

response = requests.post(url, json=data)

print(response.status_code)
print(response.json()["response"])
```

Output

```
IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bce1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/DORERAJ/AppData/Local/Programs/Python/Python313/python test86.py
200
How can I assist you today?
>>>
```

Testing Ollama API Using Python (Without Third-Party Libraries)

Overview

Testing the Ollama API using Python without third-party libraries involves using Python's built-in modules such as `urllib.request` and `json` to communicate with the Ollama server.

Purpose

The purpose of testing the Ollama API without third-party libraries is to:

- Verify that the Ollama server is running correctly.
- Learn how to interact with APIs using Python's built-in modules.
- Reduce dependency on external libraries.
- Understand the fundamentals of HTTP requests and JSON data handling.
- Build lightweight AI-powered applications.

Working Process

1. Import the built-in `urllib.request` and `json` modules.
2. Specify the Ollama API endpoint.
3. Create a dictionary containing the model name and prompt.
4. Convert the dictionary into JSON format.
5. Encode the JSON data into bytes for transmission.
6. Send an HTTP POST request to the Ollama API.
7. Ollama receives the request and processes the prompt using the selected model.
8. The API returns a JSON response containing the generated text.
9. Python decodes the response and displays the output.

Benefits

- No need to install external libraries.
- Uses only Python's standard library.
- Lightweight and portable solution.
- Helps users understand low-level API communication.
- Suitable for environments where package installation is restricted.
- Works with any Ollama-supported model running locally.

Output

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DORERAJ>cd C:\Users\DORERAJ\AppData\Local\Programs\Python\Python313

C:\Users\DORERAJ\AppData\Local\Programs\Python\Python313>dir
Volume in drive C is Windows
Volume Serial Number is CA9B-F22D

Directory of C:\Users\DORERAJ\AppData\Local\Programs\Python\Python313

12-06-2026 11:47 <DIR>      .
08-09-2025 12:06 <DIR>      ..
08-09-2025 12:07 <DIR>      DLLs
12-06-2026 11:39 <DIR>      Doc
08-09-2025 12:06 <DIR>      include
08-09-2025 12:07 <DIR>      Lib
08-09-2025 12:07 <DIR>      libs
14-08-2025 15:34          33,861 LICENSE.txt
14-08-2025 15:45      2,012,756 NEWS.txt
11-06-2026 16:58          87 python test2.py
11-06-2026 16:57          87 python test2.py.py
11-06-2026 21:48        246 python test23.py
11-06-2026 22:01        248 python test25.py
11-06-2026 16:59        137 python test45.py
11-06-2026 17:04        108 python test56.py
11-06-2026 21:28        233 python test62.py
11-06-2026 21:33        117 python test68.py
12-06-2026 11:39        261 python test86.py
14-08-2025 15:33      105,816 python.exe
14-08-2025 15:33      72,536 python3.dll
14-08-2025 15:33    6,123,864 python313.dll
14-08-2025 15:33    104,280 pythonw.exe
11-06-2026 16:32 <DIR>      Scripts
08-09-2025 12:07 <DIR>      tcl
11-06-2026 16:51        253 test.py
11-06-2026 16:51        108 test1.py
12-06-2026 11:47        261 test86.py
14-08-2025 15:34    120,400 vcruntime140.dll
14-08-2025 15:34    49,776 vcruntime140_1.dll
          20 File(s)      8,625,435 bytes
          9 Dir(s)  97,661,865,984 bytes free

C:\Users\DORERAJ\AppData\Local\Programs\Python\Python313>test86.py

C:\Users\DORERAJ\AppData\Local\Programs\Python\Python313>python test86.py
200
Hello! How can I assist you today?

C:\Users\DORERAJ\AppData\Local\Programs\Python\Python313>|
```

Testing Ollama API Using Ollama Python Library in IDLE

Overview

Testing the Ollama API using the Ollama Python Library in IDLE allows developers to interact with locally running AI models through a simple Python interface. T

Purpose

- Verify that Ollama is installed and functioning correctly.
- Simplify communication with AI models.
- Learn how to integrate AI capabilities into Python applications.
- Develop chatbots, virtual assistants, and automation tools.
- Reduce the complexity of handling API requests manually.

Working Process

1. Install the Ollama Python library using:

```
pip install ollama
```
2. Ensure the Ollama application is running and the required model is downloaded.
3. Import the chat function from the Ollama library.
4. Specify the AI model (for example, llama3.2).
5. Create a message containing the user's prompt.
6. The Ollama library automatically sends the request to the local Ollama server.
7. The selected AI model processes the prompt and generates a response.
8. The response is returned to the Python program.
9. The generated text is displayed in the IDLE shell.

Benefits

- Simple and beginner-friendly syntax.
- No need to manually handle HTTP requests or JSON data.
- Faster development and testing of AI applications.
- Supports multiple Ollama models.

Programm:

ollama test1.py - C:/Users/DORERAJ/AppData/Local/Programs/Python/Python313/ollama test1.py (3.13.7)

File Edit Format Run Options Window Help

```
from ollama import chat

response = chat(
    model='llama3.2',
    messages=[
        {'role': 'user', 'content': 'Hello'}
    ]
)

print(response['message']['content'])
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>

= RESTART: C:/Users/DORERAJ/AppData/Local/Programs/Python/Python313/ollama test1.py

How can I assist you today?

>>>

Ollama API Using Python (Without Third-Party Libraries Testing)

Overview

Ollama provides a local API that allows Python applications to interact with AI models running on your computer. By using Python's built-in libraries such as `urllib.request` and `json`, users can send prompts to Ollama and receive AI-generated responses without installing any third-party packages.

Purpose

- Verify that the Ollama server is running correctly.
- Learn API communication using Python's built-in modules.
- Reduce dependency on external libraries.
- Understand how HTTP requests and JSON data are handled.
- Develop lightweight AI applications using only the Python standard library.

Working Process

1. Import the built-in `urllib.request` and `json` modules.
2. Specify the Ollama API endpoint (`http://localhost:11434/api/generate`).
3. Create a dictionary containing the model name and prompt.
4. Convert the dictionary into JSON format.
5. Encode the JSON data into bytes.
6. Send an HTTP POST request to the Ollama API.
7. Ollama receives the request and forwards the prompt to the selected AI model.
8. The AI model processes the prompt and generates a response.
9. The response is returned in JSON format.
10. Python decodes the JSON response and displays the generated text in IDLE.

Benefits

- No additional package installation is required.
- Uses only Python's built-in libraries.
- Lightweight and efficient solution.
- Helps in understanding low-level API communication.
- Suitable for restricted environments where external packages cannot be installed.
- Works with locally hosted AI models, ensuring better privacy.
- Useful for learning REST API concepts and JSON handling.

Output

```
[notice] A new release of pip is available: 25.2 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\DORERAJ> cd C:\Users\DORERAJ\AppData\Local\Programs\Python\Python313

C:\Users\DORERAJ\AppData\Local\Programs\Python\Python313>dir
Volume in drive C is Windows
Volume Serial Number is CA9B-F22D

Directory of C:\Users\DORERAJ\AppData\Local\Programs\Python\Python313

12-06-2026  12:25    <DIR>          .
08-09-2025  12:06    <DIR>          ..
08-09-2025  12:07    <DIR>          DLLs
12-06-2026  11:39    <DIR>          Doc
08-09-2025  12:06    <DIR>          include
08-09-2025  12:07    <DIR>          Lib
08-09-2025  12:07    <DIR>          libs
14-08-2025  15:34             33,861 LICENSE.txt
14-08-2025  15:45      2,012,756 NEWS.txt
12-06-2026  12:12             181 ollama test1.py
11-06-2026  16:58              87 python test2.py
11-06-2026  16:57              87 python test2.pyy.py
11-06-2026  21:48             246 python test23.py
11-06-2026  22:01             248 python test25.py
11-06-2026  16:59             137 python test45.py
11-06-2026  17:04             108 python test56.py
11-06-2026  21:28             233 python test62.py
11-06-2026  21:33             117 python test68.py
12-06-2026  11:39             261 python test86.py
14-08-2025  15:33          105,816 python.exe
14-08-2025  15:33          72,536 python3.dll
14-08-2025  15:33      6,123,864 python313.dll
14-08-2025  15:33          104,280 pythonw.exe
11-06-2026  16:32    <DIR>          Scripts
08-09-2025  12:07    <DIR>          tcl
11-06-2026  16:51             253 test.py
11-06-2026  16:51             108 test1.py
12-06-2026  12:25             181 test2.py
12-06-2026  11:47             261 test86.py
14-08-2025  15:34          120,400 vcruntime140.dll
14-08-2025  15:34          49,776 vcruntime140_1.dll
                22 File(s)          8,625,797 bytes
                9 Dir(s)  97,700,331,520 bytes free

C:\Users\DORERAJ\AppData\Local\Programs\Python\Python313>test2.py

C:\Users\DORERAJ\AppData\Local\Programs\Python\Python313>test2.py

C:\Users\DORERAJ\AppData\Local\Programs\Python\Python313>py test2.py
Hello! How can I assist you today?
```